Ders 3 Genişletilmiş Notu: Factory Method ve Builder Kalıpları

Ders 3 Genişletilmiş Notu: Factory Method ve Builder Kalıpları

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Factory Method Kalıbı: Amaç ve Motivasyon
- 2. Belge Çerçevesi Örneğinin Çözümü
- 3. Factory Method'un Kalıp Yapısı
- 4. Kalıbın Uygulanabileceği Anlar ve Çizim Uygulaması Örneği
- 5. Abstract Factory ile Factory Method Arasındaki Fark
- 6. Örnek Gerçekleme: E-Ticaret Sitesi
- 7. Builder Kalıbı: Amaç ve RTF Okuyucu Örneği
- 8. Builder'ın Kalıp Yapısı, Bileşenleri ve Etkileşimleri
- 9. Builder'ın Uygulanabileceği Anlar, Sonuçları ve Factory Kalıplarıyla Farkı
- 10. Örnek Gerçekleme: Otomobil Bayisi
- Örnek Sorular
- Dersin Toparlaması

1. Factory Method Kalıbı: Amaç ve Motivasyon

Ders, geçen hafta görülen Abstract Factory'den sonra yaratımsal (creational) kalıplardan ikincisi olan Factory Method ile başlar.

1.1 Amaç

Slayttaki tanıma göre (ekrandan, 00:10:00) kalıbın amacı şudur: bir nesne oluşturmak için öyle bir arayüz sunmak ki, oluşturulacak nesnenin sınıfına bu arayüzü sağlayan sınıfın alt sınıfları karar verebilsin. Yani nesne oluşturma işleminin imzası üst sınıfta durur, hangi somut sınıftan nesne oluşturulacağı ise alt sınıflarda belirlenir.

1.2 Örnek problem: belge çerçevesi

Uygulama geliştirmeyi kolaylaştıracak bir çerçeve program (framework) hazırladığımızı düşünelim. Bu çerçeveyi kullanan başka programcılar, farklı belge türleriyle çalışabilecek uygulamalar geliştirecek: metin, resim ya da ikisi karışık belgeler; hem Word gibi metin belgelerini hem Excel gibi çalışma sayfalarını, hem de akla gelebilecek başka türleri düzenleyebilecek uygulamalar. Hoca, bunların her biri ayrı bir program koleksiyonu olduğu için örneği çok ilerletmeyeceğini söyledi; amaç, işin bir kısım yükünü çerçeve ile kullanıcı programcının üzerinden almaktır.

Slayta göre (ekrandan, 00:10:00) temel sınıflar Belge (Document) ve Uygulama (Application) sınıflarıdır. Çerçeveyi kullanan programcı bu iki sınıftan kalıtım yoluyla yeni sınıflar türeterek kendi yazılımını hazırlar.

Sorun şudur: bir belge oluşturmak gerektiğinde (“aç”, “yeni” gibi komutlar verildiğinde) çerçeve program bu işi nasıl yapacak? Buradaki temel belirsizlik, uygulamanın kullanıcı programcının türeteceği yeni belge sınıfları hakkında önceden bilgi sahibi olamamasıdır. Bazı tahminler yapılabilir, ama olabilecek her şeyi tahmin etmek mümkün değildir; çerçeve bizim aklımıza gelmeyen bir belge türü için de kullanılabilir.

2. Belge Çerçevesi Örneğinin Çözümü

2.1 Sınıf şeması

Kalıbın önerdiği çözüm (ekrandan, 00:20:00) şöyledir. Hoca şemayı yatay bir çizgiyle ikiye böldü: çizginin üstü çerçeve (framework), altı kullanıcı programcının oluşturacağı uygulamadır.

```
Çerçeve (framework) tarafı:
  Document (soyut)          Application (soyut)
    Open()   {soyut}          CreateDocument()  {soyut}
    Close()  {soyut}          NewDocument()
    Save()                    OpenDocument()
    Revert()
  Application ◇──"docs"──► Document   (bir uygulama birden çok belgeyi yönetir)

Kullanıcı programcı tarafı:
  MyDocument  ──kalıtım──► Document
  MyApplication ──kalıtım──► Application
    CreateDocument()        (MyApplication ----> MyDocument: oluşturur)

Not (NewDocument):        Document* doc = CreateDocument();
                          docs.Add(doc);
                          doc->Open();
Not (CreateDocument):     return new MyDocument
```

Şemada italik yazılan Application ve Document soyut sınıflardır. Bir uygulama birden fazla belge yöneteceği için aralarında parça-bütün ilişkisi kurulmuştur; elmas bütünü, ok ucu parçaları simgeler.

2.2 Somut ve soyut metotların iş bölümü

Temel düşünce, ortak ve temel işleri mümkün olduğunca çerçevede gerçekleştirmektir. Belge türünden bağımsız olarak yapılabilecek işler şunlardır:

- Yeni bir belge oluşturma komutunu başlatmak.
- Belge açma komutunu başlatmak. Hoca, belgenin nasıl açılacağından değil, açma işleminin başlatılmasından söz ettiğini vurguladı: bir diyalog kutusu çıkar, dosya disk sisteminde bulunur ve açılır. Bu kısım belge türünden bağımsızdır; kullanıcı programcıyı bununla uğraştırmanın anlamı yoktur.
- Açık belge listesine yeni bir belge nesnesi eklemek.
- Kaydetme işlemini başlatmak.
- Geri alma (undo) ve yineleme (redo) işlemleri. Document sınıfındaki Revert() metodu bu anlamdadır. Hoca burada ileride görülecek Memento kalıbının bu iş için kullanılabileceğini belirtti.

Application ve Document sınıflarının bazı metotları somut, bazıları soyuttur. Somut metotlar, belge türünden bağımsız olarak yapılabilecek işlerin ayrıntılı olarak kodlandığı metotlardır. Soyut metotlar ise kullanıcı programcının alt sınıf oluşturarak gövdelerini doldurmasını beklediğimiz metotlardır.

Çalışma yönteminin ana fikri şudur: somut metotların içinde, akışın en gerekli yerlerinde soyut metotlar çağrılır. Böylece kullanıcı programcının, akışın kendi uygulaması için farklılaşması gereken adımlarına kendi kodunu yazabileceği bağlantı noktaları oluşturulmuş olur.

> **Ek bilgi (derste anlatılmadı):** Somut bir metodun, adımlarından bazılarını soyut metotlara bırakarak bir algoritmanın iskeletini tanımlaması, GoF’taki Template Met-

hod kalıbının özüdür. GoF'a göre factory method'lar çoğu zaman bu tür şablon metotların içinden çağrılır. Hoca Template Method adını Builder anlatımında kısaca andı (01:08:10); kalıbın kendisi ileride işlenecektir.

2.3 NewDocument'ın akışı

NewDocument() önce soyut CreateDocument() metodunu çağırır. Bu metot, MyApplication alt sınıfında somutlaşır: belge türüne özgü gerekli işlemler yapılıp `new MyDocument` döndürülür. Dönen nesne MyDocument türündendir, ama MyDocument, Document'ın alt sınıfı olduğu için Document beklenen yerde kullanılabilir. Bu, kalıtımın ve çok biçimliliğin gücünün kullanılmasıdır.

Sonra yeni belge, açık belgeler listesine (`docs.Add(doc)`) eklenir ve belgenin kendisini açması için Open() metodu çağrılır. Open() ve Close() da Document'ta soyuttur ve MyDocument'ta gövdeleri tanımlanmak zorundadır. Belgelerde ortak yapılacak işler soyut Document sınıfında, farklılaşan işler onun somut alt sınıfında yürütülür. Uygulama, farklı belge türlerinin ortak işlerini ortak soyut ebeveyn üzerinden yürütebilir.

Hoca, şemadaki elmasın üzerine yazılan "docs" gösteriminin güncel UML'de de desteklenen bir tarz olduğunu söyledi. "docs", çoklu sahiplik ilişkisini modelleyen bir veri yapısıdır; büyük olasılıkla bir bağlı liste, ya da arka planda bir dizi gibi düşünülebilir. Add, dil kitaplıklarındaki veri yapılarının add, search, remove gibi hazır metotlarından biridir; ayrı bir metot olarak düşünülmemelidir.

2.4 Diğer seçenekler

Factory Method yerine başka ne kullanılabilir? Slayta göre (ekrandan, 00:20:00) bir seçenek Java'daki generic sınıflardır: `class Document<DocType>`. Bunlar C++'taki template sınıflara benzer. Bir çözüm generic olarak DocType'a bağlı oluşturulabilir.

Bu çözümün zayıf yönü dile özel olmasıdır ve uygulamanın taşınabilirliğini azaltır. C++'ta benzer yapılar olduğu için uyarlanabilir, ama bu kolay olmaz; farklı dillerdeki yapılar farklı tasarım kararlarıyla kurulmuştur ve bu farklılıkları toparlamak epey uğraştırır. Hocaya göre dile özel yetenekleri kullanmak yerine nesneye yönelik çözümler öneren tasarım kalıpları üzerinden gitmek her zaman kodun taşınabilirliğini iyileştirir ve böylece bakım masraflarını düşürür. Slayttaki "Başka?" sorusu için hoca öğrencilerden geri bildirim beklediğini söyledi.

3. Factory Method'un Kalıp Yapısı

3.1 Bileşenler

Kalıp yapısı (ekrandan, 00:21:40) şöyledir:

```
Product (soyut)                 Creator (soyut)
   ▲                              FactoryMethod()   {soyut}
   |                              AnOperation()  -> ... product = FactoryMethod() ...
ConcreteProduct  ◄------------  ConcreteCreator
                                  FactoryMethod()  -> return new ConcreteProduct
```

Önceki örnekteki Application sınıfı burada Creator'a karşılık gelir: bir ürün ortaya çıkarmaya yarayan, soyut bir sınıftır. Nesne oluşturma metodu (FactoryMethod) da soyuttur, çünkü farklı ürünler oluşturulabilir ve her birinin oluşturulma yöntemi kendi alanında çok değişebilir; bu işi soyut metot olarak bırakmak en mantıklısıdır. Creator, bu soyut metodu kullanan çeşitli işlem (operation) metotlarına sahiptir. Somut ConcreteCreator sınıfı Creator'dan, somut ConcreteProduct sınıfı Product'tan türetilir. Belirli bir ConcreteCreator belirli bir tür ConcreteProduct üretir.

3.2 İlişkiler kalıbın zorunluluğu değildir

Creator'dan Product'a bir sahiplik ya da parça-bütün ilişkisi olması kalıp için zorunlu değildir. Önceki örnekteki parça-bütün ilişkisi örneği biraz daha ayrıntılandırmak içindi. Oluşturulan ürünlerin oluşturucu tarafından saklanması ve idare edilmesi anlamlıysa Creator'dan

Product'a birebir, bire-çok ya da parça-bütün biçiminde sahiplik ilişkileri kurulabilir. ConcreteCreator ile ConcreteProduct arasında da kullanma ilişkisi yeterlidir; problem mantığı açısından avantajı varsa sahipliğe çevrilebilir.

Ters yönde, üründen oluşturucuya bir okun anlamı olmaz: bir ürün, hangi nesne tarafından oluşturulduğunu bilmek zorunda olmamalıdır. Bu illa gerekiyorsa, Creator factory method içinde ürüne kendi referansını (this) verebilir; factory method'a Creator türünden bir parametre eklenir. Ama somut alt sınıfın bu parametreyi doğru düzgün verip vermeyeceğinin garantisi yoktur.

3.3 Kalıplar harfiyen uygulanmaz

Hocaya göre bu, kalıbın minimal halidir. Problemin diğer ayrıntıları hangi ihtiyaçları getiriyorsa, kalıbın söylediklerinin üzerine ek ilişkiler kurulabilir: sahiplik, içi dolu ya da boş elmasla parça-bütün gibi. Kalıplar harfiyen uygulanması gereken çözümler değil, farklı problemlere göre özelleştirilmesi gereken çözüm önerileridir.

Ancak kalıbın önerdiklerinin çok azı alınıp çoğu dışarıda bırakılırsa, artık o kalıp uygulanmamış olur. Bu her zaman kötü değildir. Geçen haftaki örnekte, iyi bir nesneye yönelik tasarım ile Abstract Factory kullanılan tasarım arasında avantaj ve dezavantajlar olduğu görülmüştü; probleminizde hangisi daha yararlıysa o kullanılır.

4. Kalıbın Uygulanabileceği Anlar ve Çizim Uygulaması Örneği

4.1 Uygulanabileceği anlar

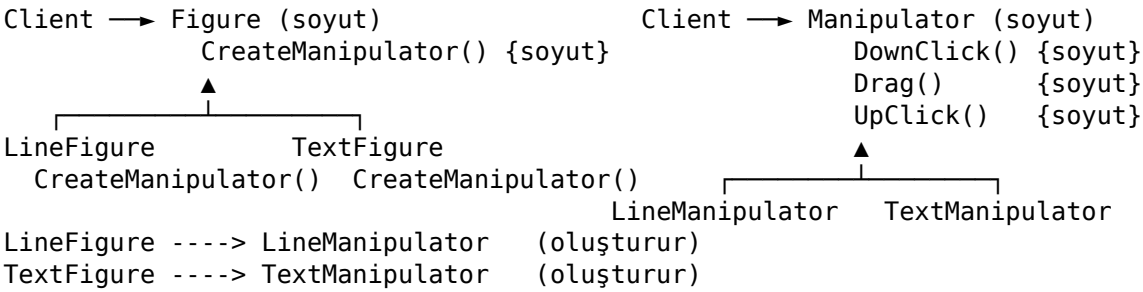
Slayta göre (ekrandan, 00:26:20) kalıp şu durumlarda uygulanabilir:

- Yeni nesneler oluşturması gereken bir sınıfın, oluşturacağı nesnenin türünü bilemediği yerlerde. Hoca buna "ya da bilmesinin istenmediği" durumları da ekledi. Kalıtım hiyerarşisi kurularak oluşturma kodlarının imzaları soyut üst sınıfta tanımlanır, gövdeleri somut alt sınıflarda kodlanır; bunlara soyut üst sınıf üzerinden erişildiği için alttaki somut sınıfların hangileri olduğunu bilmeye gerek kalmaz.
- Bir sınıfın kendi üzerindeki bir sorumluluğu çeşitli yardımcı alt sınıflarına ileteceği ve hangi tür yardımcı alt sınıfın oluşturulması gerektiği bilgisinin yerelleştirilmesi gerektiği yerlerde. Yani farklı problem durumlarında farklı yardımcıların kullanılmasının en doğru olduğu noktalarda.

4.2 Çizim uygulaması örneği

İkinci durumun örneği (ekrandan, 00:30:00): bir çizim uygulamasında şekiller fare ile değiştirilecek. Uygulamada çizgi, metin, üçgen ve dörtgen gibi geometrik şekiller olabilir. Çizgiler düz olabileceği gibi, birkaç dirsek verilebilen yuvarlak eğriler (Bezier eğrisi ya da spline) ya da sınıf şemalarındaki gibi yatay-dikey-yatay dirsekler yapan düz çizgiler de olabilir. Her şekil, soyut Figure sınıfının somut bir alt sınıfı olarak kodlanır.

Bir şeklin üstüne tıklandığında, farenin tıklama, sürükleme gibi hareketlerinin şekil türüne göre farklı etkiler yapması beklenir. Örneğin bir metin şekli tıklanıp sürüklendiğinde başka bir yere taşınır. Ama şekil bir Bezier eğrisiyse ve eğim noktalarından tutulup fare hareket ettirilirse, çizgi taşınmaz, eğimi değişir.



Soyut Figure ve soyut Manipulator sınıfları vardır. Manipulator fare olaylarını yakalayacak metotlara (DownClick, Drag, UpClick) sahiptir. Örneğin bir LineFigure'ın üstüne tıklanmışsa, işi LineManipulator yapar.

Kalıp açısından bakıldığında Creator kısmında figürler, Product kısmında manipülatörler vardır. Hoca, burada olayın tam olarak oluşturma değil kullanma olduğuna dikkat çekti; oluşturmak da düzenlemek de nihayetinde düzenleme (edit) işlemleridir. Alt sınıflarda ters yönde de bir kullanma ilişkisi görülebilir; kalıp o yöndeki bir okla genişletilebilir.

4.3 Bir varyasyon: varsayılan değiştirici

Slayta göre (ekrandan, 00:31:10) bazı şekil türleri özel bir değiştiriciye gerek duymuyorsa, Figure sınıfı varsayılan bir değiştirici örneği döndürebilir. Hoca bunu şöyle açıkladı: soyut Manipulator sınıfı somut bir üst sınıf yapılır ve özel değiştirici istemeyen şekiller bu üst sınıf manipülatörle ilişkilendirilir. Böylece her şekil için ayrı bir değiştirici sınıfı yazmaya gerek kalmaz.

5. Abstract Factory ile Factory Method Arasındaki Fark

MVC sayılmazsa, ayrıntılı çalışılan ilk iki tasarım kalıbı Abstract Factory ve Factory Method'dur. Bu iki kalıp birbirine karıştırılabilir ya da bir problemde hangisinin kullanılacağı sorusu kafa karıştırabilir.

> **Hocanın vurgusu:** Hatırlanması gereken en önemli etken şudur: Abstract Factory'de aileler ve ailelerin üyeleri vardır. Örneğin çeşitli pencere yöneticisi (window manager) aileleri ve bu ailelerin radio button, check button, input box gibi üyeleri. Factory Method'da bu yoktur; Abstract Factory'deki ayrı aile hiyerarşileri Factory Method'da bulunmaz. "Bunu unutmayın lütfen."

6. Örnek Gerçekleme: E-Ticaret Sitesi

6.1 Problem

Hoca bu problemi Head First Design Patterns kitabındaki bir örnekten esinlenerek ve onu değiştirerek hazırladı (ekrandan, 00:35:50). Bir e-ticaret sitesi farklı tür ürünler satıyor: elektronik, giyim, kırtasiye ürünleri ayrı ayrı. Sitenin yönetici çalışanları, bir müşterinin siparişindeki farklı tür ürünleri işleme koymak istiyor.

Müşteri siparişini işleme adımları belli bir sırada yürütülür, ancak adımlardaki iş mantığı ürün türüne göre çok farklıdır. Örneğin gerekiyorsa ek garanti belgesi oluşturulur; ürün uygun bir kargo firmasına verilir; ürün merkezi depoda varsa oradan kargolanır, yoksa ana depodan getirtilir; ürünü bağımsız bir dükkân sahibi satıyorsa ona stok bilgisi iletilir. Elektronik ürün için garanti belgesi hazırlanırken kırtasiye için garanti belgesine gerek yoktur. Özetle ortak yönlerden çok farklı adımlar vardır.

Amaç, site çalışanına siparişleri işlemekte kullanabileceği bir araç sağlamaktır. Hocanın düşündüğü kullanım, biraz komut verme gibidir; çalışanın hafif kodlama bildiği varsayılır. Slayttaki kullanım örneği:

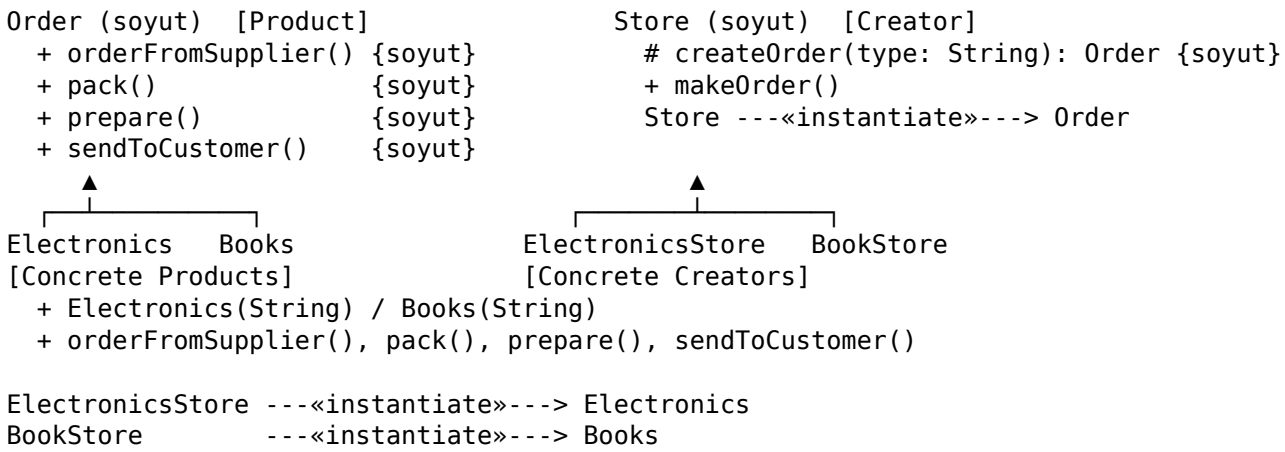
```
package factoryMethod.example;
public class MainApp {
   public static void main(String[] args) {
      Store store;
      //Elektronik ürün siparişi geldi
      store = new ElectronicsStore( );
      store.createOrder("5524345678");
      //Kitap siparişi geldi
      store = new BookStore( );
      store.createOrder("8693243565");
   }
}
```

Bir store işaretçisi yeniden kullanılır. Önce elektronik ürün siparişi gelir: elektronik mağazası türünden bir nesne oluşturulur ve ürünün barkodu verilerek iş başlatılır. Sonra bir kitap siparişi gelir: store bu kez bir kitap mağazasını gösterir ve kitabın ISBN numarası verilir.

> **Ders sonrası not:** Slayttaki kullanım örneğinde istemci `store.createOrder(...)` çağırıyor. Oysa çözümde (bölüm 6.3) createOrder() protected'dır ve yalnız sipariş nesnesini oluşturur; hazırlama, tedarik, paketleme ve gönderme adımlarını yürüten public makeOrder() metodudur. İstemcinin büyük olasılıkla `store.makeOrder(...)` çağırması amaçlanmıştır. Hoca derste bu farka değinmedi.
>
> **Ek bilgi (derste anlatılmadı):** Java'da protected üyeler hem alt sınıflardan hem de aynı paketteki sınıflardan erişilebilir. Bu yüzden MainApp, Store ile aynı pakette olduğu sürece `store.createOrder(...)` çağrısı derlenir; farklı bir paketteki istemci için derleme hatası olur.

6.2 Sınıf şeması

Çözüm şeması (ekrandan, 00:37:40) kalıbın rollerini not kutularıyla işaretler:



Store, oluşturma işlemlerini yapan sınıftır; yapılabilecek ortak işler bu soyut sınıfta tanımlanır. Siparişi oluşturmak için createOrder, siparişi gerçekten yürürlüğe koymak için makeOrder metodu vardır. Store'dan kalıtımla ElectronicsStore, BookStore ve başka hangi tür dükkânlar olacaksa onları simgeleyen alt sınıflar oluşturulur. Siparişlerin yönetileceği Order sınıfı da soyuttur. Ne işlerin yapılacağı belli, ama nasıl yapılacakları birbirinden çok farklı olduğundan Order'ın bütün metotları soyuttur. Electronics ve Books için ayrı somut alt sınıflar oluşturulur ve bütün soyut metotların gövdeleri doldurulur.

6.3 Kod

Order hiyerarşisi (ekrandan, 00:38:50):

```java
package dp.factoryMethod.example;
public abstract class Order {
    public abstract void prepare( );
    public abstract void orderFromSupplier( );
    public abstract void pack( );
    public abstract void sendToCustomer( );
}
public class Electronics extends Order {
    private String barcode;

    public Electronics( String barcode ) {
        super( ); this.barcode = barcode;
    }
    public void orderFromSupplier() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
```

```
    public void pack() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
    public void prepare() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
    public void sendToCustomer() {
        //Elektronik malzeme için yapılacak işlemlerin tanımlanması
    }
}
public class Books extends Order { /*Coded similarly*/ }
```

Hocanın ifadesiyle kodda "çok enteresan bir şey yok". Elektronik eşya siparişini simgeleyen somut alt sınıf, barkodu alır, üst sınıfın kurucusunu çağırır ve barkodu atar; sonra dört işlemi elektronik ürüne göre gerçekler. Kitap siparişi de aynı biçimde davranır.

Store hiyerarşisi (ekrandan, 00:40:00):

```
public abstract class Store {
    protected abstract Order createOrder( String uniqueID );
    public void makeOrder( String uniqueID ) {
        Order newOrder = createOrder( uniqueID );
        newOrder.prepare( );
        newOrder.orderFromSupplier( );
        newOrder.pack( );
        newOrder.sendToCustomer( );
    }
}
public class ElectronicsStore extends Store {
    protected Order createOrder( String uniqueID ) {
        Electronics newElectronicsOrder = new Electronics(uniqueID);
        //Elektronik malzeme siparişi için yapılacak işlemlerin tanımlanması
        return newElectronicsOrder;
    }
}
public class BookStore extends Store {
    protected Order createOrder(String uniqueID ) {
        Books newBookOrder = new Books(uniqueID);
        //Kitap siparişi için yapılacak işlemlerin tanımlanması
        return newBookOrder;
    }
}
```

Store, kalıbın "fabrika" sınıfıdır. createOrder'ın imzası soyut olarak tanımlanır. Somut olarak yapılacak ortak işler ise makeOrder'dadır: sipariş oluşturulduktan sonra hazırlanması, tedarikçiden temin edilmesi, paketlenmesi ve müşteriye gönderilmesi. Farklılaşan adım oluşturma olduğu için createOrder soyut bırakılır; somut Store sınıflarında bu metot "ete kemiğe büründürülür".

Hoca bu örneğin alternatif çözümlerin karşılaştırıldığı bir örnek olmadığını, yalnız kalıbın uygulanma şeklini gösterdiğini belirtti.

7. Builder Kalıbı: Amaç ve RTF Okuyucu Örneği

Aradan sonra yaratımsal kalıplardan Builder'a geçildi. Hoca bu bölümün başında ekran paylaşımını slayta geçirmeyi unuttuğu için girişteki slayt kayıtta görünmüyor; aşağıdaki amaç ve problem tanımı hocanın sözlü anlatımından alınmıştır.

7.1 Amaç

Elimizde karmaşık bir varlık vardır ve bu varlık, çeşitli alt parçalarının farklı biçimlerde oluşturulup birleştirilmesiyle elde edilir. Bunu kolaylaştırmak için temel ilke yine "neyin değiştiğini

bul ve onu sarmala"dır. Builder'daki amaç, karmaşık bir nesnenin oluşturulmasını gösteriminden ayırmaktır; böylece aynı oluşturma işleminden farklı gösterimler elde edilebilir. Hoca, bunun ilk duyulduğunda şaşırtıcı geldiğini, ama sınıf şemasına bakıp koda dökünce olmayacak bir şey olmadığının görüleceğini söyledi.

7.2 GoF örneği: RTF okuyucu

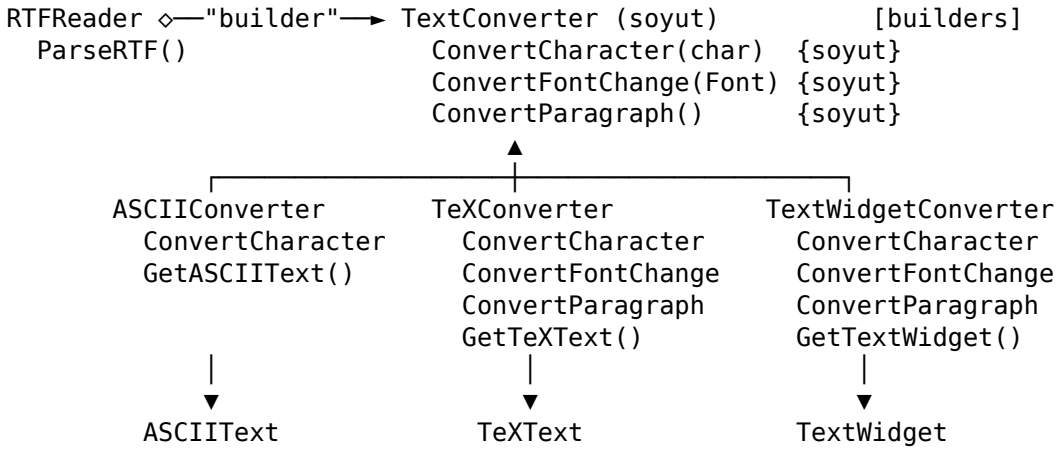
Dörtlü çetenin önerdiği örnek bir metin okuyucu programıdır. Belge Rich Text Format (RTF) biçimindedir; bu biçim yalnız metinsel değil, 2 boyutlu grafik, ses, video gibi bileşenler de içerebilir (basit bir metin olsaydı sorun olmazdı). Program, açtığı belgeyi farklı dosya ve metin türlerine, hatta bu metni düzenleyebilecek bir GUI bileşenine çevirebilmelidir.

Sorun şudur: çok farklı metin türleri olacak ve bunların hepsini hemen gerçeklemek zorunda değiliz, adım adım gideceğiz. Bu, işimizi kolaylaştırır ama tasarımı zorlaştırır, çünkü ileride yeni metin türleri eklemek mutlaka gerekecektir. Bu işin çok zor olmadığı bir çözüm istenir.

Dörtlü çetenin çözüm önerisi şöyledir: metni okumakla yükümlü bir sınıf, bir biçim çevirici nesnesine çeviri isteği gönderir. Çevirici nesne hem asıl çevrim işinden hem de çevrilen metni belli bir biçimde simgelemekten sorumludur. Böylece yeni metin türü eklemek için yalnız yeni bir çevirici sınıf eklemek gerekir. Bunu mevcut altyapıya bağlamak için kalıtım kullanılır; en üstte somut değil soyut bir çevirici bulunur ve farklı metin türü çeviricileri onun alt sınıfları olarak eklenir.

7.3 Çözümün şeması

Çözüm şeması (ekrandan, 01:00:00):




```
ParseRTF() notu:
while (t = get the next token) {
    switch t.Type {
    CHAR:
        builder->ConvertCharacter(t.Char)
    FONT:
        builder->ConvertFontChange(t.Font)
    PARA:
        builder->ConvertParagraph()
    }
}
```

RTFReader ana uygulamadır. TextConverter soyut sınıfını kullanır ve yönlendirir; iş ParseRTF() metodunun içinde yürütülür. Hoca, şemada elmas (parça-bütün) gösterilse de bunun belki birebir bir ilişki olabileceğini söyledi: çalışma anında setBuilder gibi bir metot çağrılarak RTFReader'a farklı somut çevirici sınıflarından birinin nesnesi verilebilir. Ya da RTFReader farklı çeviricileri zaten bir liste içinde tutuyor olabilir.

Her metin türü için ayrı bir somut çevirici vardır: düz metin için ASCII, biçimli metin için TeX, metni düzenlemeyi sağlayacak bir GUI bileşeni oluşturmak için TextWidgetConverter. RTF-

Reader'ın bunların herhangi biriyle çalışabilmesi için soyut bir üst sınıfın bulunması gerekir; yapılacak iş, o soyut metotlar üzerinden somut gerçeklemelere iletilir.

> **Hocanın vurgusu:** Bu şema GoF kitabından alınmıştır ve kitap yazıldığında UML henüz bir standart olarak önerilmemişti. Kalıtım okunun ucu üst sınıftan uzakta çizilmiştir; güncel standarda göre okun ucu üst sınıfa değmelidir. Soldaki gibi çizilirse karga ayağı gösterimiyle karışabilir. Sınavlarda kesinlikle öyle yapılmamalı, çizimler güncel UML standardına göre yapılmalıdır.

7.4 ParseRTF'in çalışması

Karmaşık ürünü oluşturan parçaların her biri bir token'dır. Token'ların bir dosyadan mı okunduğu, internet üzerinden birer birer mi indirildiği önemli değildir. Her t token'ı alınır ve türüne bakılır:

- Karakter token'ı ise (CHAR) builder'ın ConvertCharacter metodu, token'daki karakter parametre olarak verilerek çağrılır.
- Font değişikliği komutu ise (FONT) ConvertFontChange metodu, gelen font verilerek çağrılır.
- Yeni paragraf komutu ise (PARA) ConvertParagraph metodu çağrılır. Burada ek bilgiye gerek yoktur; ama paragrafın önce-sonra boşluk, satır aralığı gibi bir biçimi varsa, `t.Para` biçiminde parametre de verilebilir.

Hoca, slayttaki case yazımının pseudo code gibi düşünülmesi gerektiğini söyledi. ASCII çevirici ASCII metin, TeX çevirici TeX metni, widget çevirici widget oluşturur; builder'ın farklı somut alt sınıfları kendisiyle ilişkili türden bir ürün üretir.

7.5 Çözümün eleştirileri

Hocaya göre bu çok iyi bir çözümdür, ama iki yönden eleştirilebilir. Slayttaki eleştiriler (ekrandan, 01:00:00):

- Çok biçimliliği tam kullanamadık, switch-case hâlâ duruyor.
- Alt sınıflar üst sınıfın tüm metotlarını gerçeklemelidir. Bu yüzden alt sınıfta anlamı olmayan metotlar boş gövdeli olarak kalmak zorundadır.

Hoca önce dördüncü bir biçim (örneğin PDF) eklemeyi düşündü, sonra bunun asıl sorun olmadığını belirterek kendini düzeltti: yeni bir PDFConverter alt sınıfı eklemek sorun değildir. Asıl sorun, ortaya çıkacak üründe yeni bir alt bileşen türü olmasıdır. Örneğin belgede gömülü video (ya da çalıştırılabilecek gömülü bir betik) olabilsin. Bunu ele almak için soyut TextConverter'a bir ConvertVideo metodu koymak gerekir. Bu metot konur konmaz, daha önceden var olan bütün alt sınıflarda da bu metodun gövdesinin tanımlanması gerekir; oysa onlarda video bileşeninin hiçbir anlamı yoktur. Sonuçta boş gövdeli metot kodlanır, yoksa derleme hatası alınır. RTFReader'daki switch'e de yeni bir case eklenir.

Aynı sorun bugünkü şemada da görülür. ASCIIConverter'ın karakter çevirmesi mantıklıdır, ama düz metinde font değişimi desteklenmez. Paragraf biçimi de desteklenmez; düz metinde paragraf yalnız yeni satır karakteri koymaktan ibarettir. Hoca, ConvertParagraph'ın bu nedenle belki eklenebileceğini, ama ConvertFontChange'in hiçbir anlamı olmayacağını söyledi.

Yani bu kalıpta yeni bir alt bileşen türü eklemek çok zordur ve switch-case'ten tamamen kurtulunamamıştır. Hoca bunun bir kod kusuru (code smell) olarak kendini gösterdiğini söyledi: hata değildir, ama esnekliği azaltır ve nesneye yönelimin önerdiği makul kullanım biçimlerinin dışında kalır. Yine de hiç çözümün olmamasından ya da daha karmaşık bir çözümden iyidir; bizi bir yere kadar götürür.

8. Builder'ın Kalıp Yapısı, Bileşenleri ve Etkileşimleri

8.1 Kalıp yapısı

Kalıp yapısı (ekrandan, 01:10:00):

```
Director ◇—"builder"—► Builder (soyut)
  Construct()              BuildPart() {soyut}
```

```
                    ▲
ConcreteBuilder ----> Product
  BuildPart()
  GetResult()
```

```
Construct() notu:
for all objects in structure {
    builder->BuildPart()
}
```

Director, karmaşık varlığın oluşturulmasıyla ilgili adımları koordine eden ve yöneten sınıftır. Somut bir Construct metodu vardır; yapıdaki her parça için ilişkili Builder nesnesinin "al bu parçayı, bütüne dahil et" metotlarını çağırır. ConcreteBuilder, BuildPart metodunun (ya da metotlarının) gövdesini doldurur ve kendisiyle ilişkili türden ürünü döndürmek için GetResult gibi bir metoda sahiptir.

8.2 Switch-case nerede olmalı?

Slayttaki açıklamaya göre (ekrandan, 01:10:00) kalıbın bu tasarımında switch-case gözükmüyor, ama ConcreteBuilder.buildPart metodunda kendini gösterecektir. Çünkü Director.construct metodundaki "for all objects in structure" döngüsündeki nesneler bir şekilde buildPart metoduna parametre olarak aktarılmak zorunda kalınacak ve "parça şu ise şunu yap, bu ise bunu yap" gibi komutlar verilmek zorunda kalınacaktır. Slayta göre birden fazla somut Builder.buildPart içinde switch-case kullanmaktansa Director.construct içinde tek switch-case kullanmak daha iyidir.

Hoca bunu şöyle açıkladı: switch-case, kimi vakalarda Director'da, kimi vakalarda ConcreteBuilder'da olabilir. Somut alt sınıflara atanırsa, Director yalnız token'ı verir ve her somut builder "token karakterse şöyle, fontsa böyle yap" diye hareket eder. Ama somut alt sınıfların sayısı fazla, Director ise bir tanedir. Switch-case'i olabildiğince az yerde kullanmak için bu işi Director'da yapmak daha doğrudur. Director "bütün token'ları işle" biçimindeyse iş mecburen ConcreteBuilder'a düşer. Soyut Builder sınıfına taşımak da mümkündür, ama o zaman yapı Template Method'a dönüşür; hoca buna girmedi.

8.3 Kalıp bileşenleri

Slayttaki bileşen tanımları (ekrandan, 01:10:20):

- **Builder:** Ürün nesnesinin parçalarını oluşturmak için bir arayüz sunar. Hoca, bunun tek metot gibi göründüğünü, ama örnekteki gibi birkaç metot olabileceğini söyledi.
- **Product:** Ürün. Kendisini oluşturan parçaları tanımlayan sınıfları içerir. Parçaları bir bütün haline getirmek için gerekli arayüzleri (komutları) içerir; hocaya göre bunların Product'ta yer alması en uygunudur.
- **Director:** Ürünü, Builder arayüzünü kullanarak oluşturur. Hoca, Builder soyut sınıfının bütün metotları soyutsa onun Java'nın interface yapısıyla da tanımlanabileceğini söyledi.
- **ConcreteBuilder:** Builder gerçeklemesi; bir ya da birden fazla kardeş sınıf olabilir. Ürün nesnesinin parçalarını oluşturur ve birleştirir; işin nasıl yapılacağı burada kodlanır. Oluşturacağı gösterimi tanımlamak ve oluşturma işlemini izlemekle yükümlüdür. Oluşan ürünü Director'a geri vermek üzere bir getter metodu sunar.

8.4 Bileşenlerin etkileşimleri

Sıralama şeması (ekrandan, 01:11:50) şu akışı gösterir:

```
aClient                      aDirector               aConcreteBuilder
   |-- new ConcreteBuilder ----------------------------------> |
   |-- new Director(aConcreteBuilder) -->|                     |
   |-- Construct() --------------------->|                     |
   |                                     |-- BuildPartA() ---->|
   |                                     |-- BuildPartB() ---->|
   |                                     |-- BuildPartC() ---->|
   |-- GetResult() ------------------------------------------ > |
```

İstemci önce bir ConcreteBuilder oluşturur (hangi somut türden oluşturacaksa). Sonra bir Director oluşturur ve ona bu builder'ı verir. Ardından Director'ın Construct metodunu çağırır. Director A, B, C kısımlarının oluşturulması komutlarını verdikten sonra, istemci GetResult metoduyla tamamlanmış ürünü alır.

Hoca, GoF kitabından alınan sıralama şemalarında dönüş oklarının pek çizilmediğini belirtti. Dönüş okları tam çizilmezse, bir metodun diğerinin bitiminden sonra mı yoksa içinde mi yürütüldüğü net anlaşılmaz. Bu şemada şöyle okunmalıdır: new ConcreteBuilder çalışıp tamamen bittikten sonra Director nesnesi oluşturulur; o da döndükten sonra iki nesne de elde ve birbiriyle ilişkilidir. Sonra Construct çağrılır. BuildPartA yürütülüp biter (kontrol çubukları arasındaki boşluktan dönüş okunun var olduğu çıkarılabilir), sonra BuildPartB, ancak o bittikten sonra BuildPartC yürütülür. Bütün alt parçalar bitince Construct sona erer.

Construct metodu sonucu doğrudan geri döndürmez; ürünü döndürmek için ayrı bir getter metodu olması daha anlamlıdır. GetResult çağrısının dönüş okunun üstüne "product" yazılabilir. Hoca bunları derste şemanın üzerine elle yazarak gösterdi.

9. Builder'ın Uygulanabileceği Anlar, Sonuçları ve Factory Kalıplarıyla Farkı

9.1 Uygulanabileceği anlar

Slayta göre (ekrandan, 01:16:10):

- Karmaşık bir nesneyi oluşturma algoritmasının, nesneyi oluşturan parçalardan bağımsız olması ve bu parçaların nasıl birleştirileceğinden bağımsız olması gerektiği anlarda.
- Bir nesneyi oluşturma sürecinin, bu nesnenin farklı gösterimlerinin olmasına izin verebilmesi gerektiği anlarda.

Hoca bunu şöyle özetledi: Director kendisine verilen Builder'a "bak bu parçalardan al, bunları yap" der; Builder ilgili ürünü oluşturur; sonra ürün getter metoduyla oluşturucudan alınır. Malzemeleri verir, "bunu yap" der, geri alır. Bu sırada Director'ın neyin nasıl yapılacağını bilmesi gerekmez.

9.2 Sonuçlar

- Ürünün iç yapısının en az yan etkiyle değiştirilebilmesi mümkün olur (yeni bir ConcreteBuilder sınıfı yazarak). Hoca burada "iç yapı" yerine "farklı gösterim biçimleri" demenin daha doğru olacağını söyledi. İç yapıda değişiklik olduğunda (örneğin yeni bir parça türü eklendiğinde) Director'daki case'e yeni bir dal eklenir; bu bir yan etkidir ama aşırı büyük değildir. Alternatifi, bu işi her bir Builder'a eklemektir ki bu "daha kabus bir iş"tir. Özetle ürüne farklı bir gösterim eklemek kolay, iç yapısını değiştirmek biraz daha zordur.
- Önceden görülen "factory" kalıplarıyla karşılaştırıldığında ürün oluşturma süreci üzerinde daha fazla denetim sahibi olunur: Director örneği, Builder örneğine hangi işlemi hangi sırada yapması gerektiği emrini verir.

9.3 Factory kalıplarıyla farkı

MVC sayılmazsa Builder dördüncü, yalnız yaratımsal kalıplar açısından bakılırsa üçüncü kalıptır (Abstract Factory ve Factory Method'dan sonra). Hoca, karıştırmamak için şunun düşünülmesini istedi:

> **Hocanın vurgusu:** Builder'da ürünün alt parçalarının oluşturulup ürüne katılmasında daha fazla denetimimiz vardır. Factory kalıplarında istemci "bana şu üründen ver" der, ister ve alır. Builder'da Director, parçaların gönderilmesinde daha fazla denetim sahibidir; isterse belirli alt parçaları Builder'a göndermemeyi seçip bir yarı mamul oluşturabilir. Abstract Factory'de ve Factory Method'da durum böyle değildir. Abstract Factory'nin ayırt edici yönü de ürün aileleri ve o ailelerin bireyleri olan ürünlerdir. "Bunları lütfen unutmayın."

10. Örnek Gerçekleme: Otomobil Bayisi

Bu örnek hocanın kendisine aittir, bir kitaptan alınmamıştır. Hoca, bir örneğin kimden olduğunun şemadan anlaşılabileceğini söyledi: hazır bir kitaptan örnek aldıysa kitaptaki şemanın ekran görüntüsünü alıp notlarını ekler; kendisi oluşturuyorsa şemaları sıfırdan Violet UML ile çizer.

10.1 Problem

Bir otomobil bayisi (ya da oto galeri) farklı markaların birden ortak bayiliğini yapıyor. Hocaya göre bayi olmak daha mantıklı bir senaryodur; günlük hayatta da bir X firması A, B ve C markalarının ürünlerini ayrı binalarda satabilir, ama hepsinin ortak çatı şirketi aynıdır.

Slayta göre (ekrandan, 01:20:00): farklı marka ve model araçların satıldığı bir bayide, markalara özel adlandırma ifadelerine rağmen ortak bir araç modellemesi yapılacak. Araçlar kasa, yakıt türü, motor hacmi ve gücü gibi bileşenlerin bir araya getirilmesi ile karmaşık bir süreçle oluşmaktadır. Amaç süreçleri ve otomasyon yazılımını kolaylaştırmaktır.

> **Hocanın vurgusu:** Yazılım sürece uymalı, süreç yazılıma uydurulmamalıdır. Hoca, hazır bir ürün alıp çalışma biçimini ona uyacak şekilde değiştirmenin sık yapılan bir yanlış olduğunu söyledi.

Araç bileşenleri şunlardır:

- **Kasa:** sedan, hatchback, station wagon, SUV.
- **Yakıt türü:** benzin (petrol), dizel, LPG, elektrik. Hoca, bunu hazırladığı zamanın eski olduğunu, bugün elektrik denince plug-in hibrit, tam hibrit ya da saf elektrik gibi alt seçenekler sorulacağını söyledi; şimdilik tek seçenek olarak tuttu ve genişletmenin ne kadar kolay olacağının ayrıca tartışılabileceğini belirtti.
- **Vites (transmission):** ileri vites sayısı (4, 5, 6, 7 gibi) ve otomatik olup olmadığı.
- **Motor:** motor hacmi ve beygir gücü.

Bu bileşenlerin bir araya getirilmesi fabrikada olduğu gibi yazılımda da karmaşık bir süreçtir. Üstelik farklı markalar, farklı modeller bu temel bileşenleri farklı anahtar kelimelerle ifade eder. Örneğin Honda'da model adında CR-V geçiyorsa o aracın SUV'si olduğu anlaşılır; Peugeot'ta ise ürün kodunun basamak sayısı kasa türü hakkında bilgi verir. Bütün bunların koda dökülmesi ve aracın uzun, ayrıntılı adından vites, motor gibi bileşenlerin çıkarılabilmesi gerekir.

Karmaşık parçalardan oluşan bir ürünü oluşturmak ve idare etmek için bir tasarım gerektiğinde, hocaya göre Builder kalıbı hemen akla gelir; bu problem tam da kalıbın kullanılabileceği problemlerden biridir.

10.2 Ürün paketi

product paketi (ekrandan, 01:20:00):

```
«enum» Chassis:   SEDAN, HATCHBACK, STATIONWAGON, SUV
«enum» FuelType:  PETROLEUM, DIESEL, LPG, ELECTRIC

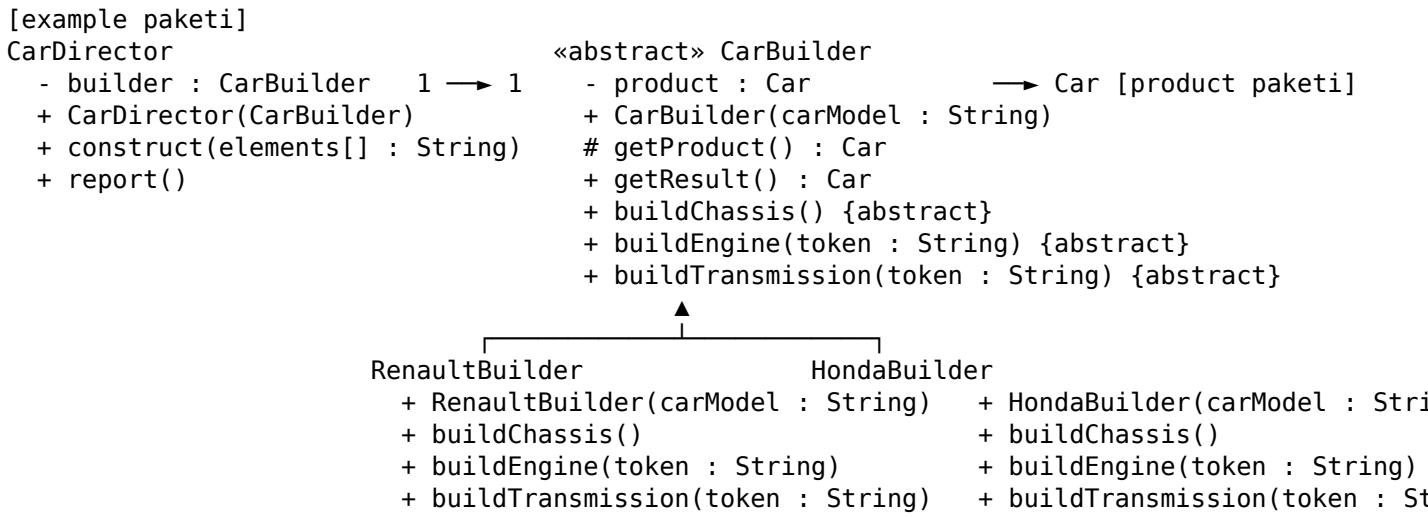
Transmission
  - gearCount : int {final}
  - isAutomatic : boolean {final}

Car
  - brand, model : String
  Car ──► Transmission
  Car ──► Chassis
  Car ──► Engine

Engine
  - volume, power : int
  Engine ──► FuelType
```

10.3 Önerilen çözüm

Builder kalıbına uygun olarak yapılan tasarım (ekrandan, 01:30:00):



Hoca, problem alanındaki varlıkları adlandırırken kalıptaki kelimeleri (Director, Builder) bilerek kullandığını söyledi. Böylece problemdeki hangi bileşenin kalıptaki hangi bileşene denk geldiği rahatça görülür; aksi halde iki sınıf şemasını yan yana koyup göz ile eşleştirmek gerekir. Bu, teknik personel için yararlıdır. Son kullanıcı "Builder", "Director" kelimelerini anlamayabilir, ama zaten koda ya da teknik belgelere bakmayacaktır. Kalıp kullanılan çözümlerde, problem alanındaki adın yanına kalıptaki rolün adını eklemek önerilir.

CarDirector, somut marka builder'ı ile eşleşir. Construct metodu bir string dizisindeki elemanları alıp aracı oluşturur. Hoca burada "biraz kolaya kaçtığını" söyledi: tek bir uzun string alıp token'lara bölmek ve token'ları tek tek işlemek yerine, kalıba odaklı daha basit bir kod için string önceden parçalara bölünmüş olarak verilir. report() metodu, oluşturulan aracı getResult ile alıp ekrana yazdırır. CarBuilder'da oluşturulacak ürünün bir üye alan olarak durması zorunludur; aracın ana modeli kurucuya tek parametre olarak gelir.

10.4 Director ile Builder arasındaki iş bölümü

Hoca önce bu örnekte bir tür denetimin tersine çevrilmesi (inversion of control) yaptığını, GoF örneğinin aksine Director'ı basit, Builder'ı ayrıntılı tuttuğunu ve elemanların tek hamlede Builder'a verildiğini söyledi. Gerekçesi şuydu: farklı marka ve modellerle çalışmak istendiğinde, değişen şey belirli bir markanın yeni bir modelinin çıkması ya da adlandırma kuralının değişmesidir (örneğin daha önce SUV satmayan bir marka SUV satmaya başlar ve bunun için yeni bir anahtar kelime kullanır). Değişiklik orada olduğuna göre, değişiklik yalnız o markanın builder'ında yapılmalıdır. Her aracın bir kasası vardır; gelen token'ın kasayla ilgili olduğu kolayca anlaşılsaydı bu iş Director'da yapılabilirdi. Ama işler markaya göre değiştiği için token'ın neye ait olduğu ve nasıl işleneceği bilgisinin ona en çok vakıf olan somut marka builder'ına konması daha mantıklı geldi.

Kodu açınca hoca bunu geri aldı (01:37:39): ayrıntılı token kontrolü yine Director'ın içindedir. Director her elemana bakar; motor içeriyorsa motoru, vites içeriyorsa vitesi oluşturur. Bütün token'lar olduğu gibi Builder'a gönderilmez. Markaya özgü ayrıntılar ise ilgili Builder sınıflarında kodlanır; örneğin kasa oluşturulurken model adında "back" geçiyorsa hatchback, başka bir anahtar kelime geçiyorsa SUV, hiçbiri geçmiyorsa varsayılan olarak sedan seçilir. Böylece Director ile Builder'lar arasında bir sorumluluk bölümü atanmış olur.

10.5 CarDirector kodu

CarDirector sınıfının kodu (ekrandan, 01:43:20, 01:48:30):

```
private CarBuilder builder;

public CarDirector(CarBuilder builder) {
```

```
    this.builder = builder;
}

public void construct(String [] elements) {
    builder.buildChassis();
    for( String element : elements ) {
        int tab = element.indexOf("\t");
        String item = element.substring(0,tab);
        String info = element.substring(tab+1,element.length());
        if( item.compareToIgnoreCase("Engine")== 0 )
            builder.buildEngine(info);
        else if( item.compareToIgnoreCase("Gear")== 0 )
            builder.buildTransmission(info);
    }
}
public void report( ) {
    System.out.println(builder.getResult());
}

public static void main(String[] args) {
    CarDirector director = new CarDirector( new HondaBuilder("Civic Sedan") );
    String elements[] = {"Engine\t125PS","Engine\t1.6L","Gear\t6AT"};
    director.construct(elements);
    director.report();
}
```

construct önce şasiyi (kasayı) oluşturur; kasanın anlamı marka-model adının içinde geçer. Beygir gücü, motor hacmi gibi bilgiler ise ayrıntılı token'lar olarak gönderilir. Her eleman, sekme (\t) ayracıyla solda bileşen türü (item), sağda değer (info) olmak üzere ikiye bölünür. Item "Engine" ise builder'ın buildEngine metodu, "Gear" ise buildTransmission metodu ilgili değerle çağrılır. En sonunda report çalıştırılıp ürün yazdırılır.

Örnek araç Honda Civic Sedan'dır. Token'larda PS beygir gücü, L litre cinsinden motor hacmi, AT ise otomatik vites anlamındadır: "6AT" altı ileri otomatik vites demektir. Hoca, başka bir araç için de örnek bir satır eklenmesinin daha açıklayıcı olacağını söyleyerek koda bir not düştü.

10.6 CarBuilder ve getResult'un klon döndürmesi

CarBuilder sınıfının başı (ekrandan, 01:40:00):

```
package dp.builder.example;

import dp.builder.product.*;

public abstract class CarBuilder {
    private Car product;
    public CarBuilder( String carModel ) {
        product = new Car(carModel);
    }
    public Car getResult() {
        if( product == null )
            return null;
        Car clone = new Car(product.getModel());
        clone.setBrand(product.getBrand());
        clone.setChassis(product.getChassis());
        clone.setEngine(product.getEngine());
        // ... (devamı ekranda görünmüyor)
```

Kasa, motor ve vites için build metotları soyuttur, çünkü farklı marka ve modeller için farklı biçimde kodlanırlar.

Hoca, getResult'un ürünü doğrudan değil klonlayarak döndürmesinin nedenini kodu görünce hatırladı. CarBuilder'ın product üye alanı doğrudan döndürülüp kullanılsaydı, döndürülen nesne üzerinde değişiklikler yapılabilir, olmayan bir motor ya da vites bileşeni atanabilirdi. Nesne referansı korunduğu için ürün yanlış çalışacak biçimde manipüle edilebilirdi. Setter yazılmasa bile, üye alan olan referansın kendisinin döndürülmesi, o referans üzerinde değişiklik kodlarının yürütülmesine engel olmaz. Klon döndürülürse, klonda yapılan değişiklik asıl ürünü etkilemez.

Bu yüzden iki ayrı metot vardır: alt sınıfların product üye alanına erişebilmesi için protected `getProduct(): Car`, oluşturma işi bittikten sonra sonucu almak için ise ürünün kopyasını döndüren public `getResult(): Car`. getProduct, başkası kafasına göre erişemesin diye protected tutulmuştur.

10.7 HondaBuilder kodu

HondaBuilder'ın kurucusu üst sınıfın kurucusunu çağırır ve marka alanına "Honda" yazar, çünkü bu builder Honda araçları üretecektir. Kasa oluşturulurken az önce anlatılan kurallara göre ilgili token yorumlanıp kasa türüne karar verilir. buildEngine metodu (ekrandan, 01:45:40):

```
public void buildEngine(String token) {
    //Min. values for a Honda engine
    if( getProduct().getEngine() == null )
        getProduct().setEngine( new Engine(FuelType.PETROLEUM, 1150, 60) );
    token = token.toUpperCase();
    if( token.contains("ECO") )
        getProduct().getEngine().setFuel(FuelType.LPG);
    else if( token.contains("PS") ) {
        int end = token.indexOf("PS");
        token = token.substring(0, end);
        getProduct().getEngine().setPower( Integer.parseInt(token) );
    }
    else if( token.contains("L") ) {
        // ... (devamı ekranda görünmüyor)
```

Motorla ilgili özel bir bilgi verilmemişse varsayılan değer kullanılır: benzinli, 1150 cc (1,15 litre) motor hacmi ve 60 beygir gücü. Token "PS" içeriyorsa beygir gücü anlamındadır; "PS"den önceki sayısal değer ayrıştırılıp (parse) motorun gücü olarak yazılır. Token "L" ile bitiyorsa motor hacmiyle ilgilidir; sayısal değer alınıp motora yazılır. Vites için de benzer biçimde varsayılan kurallar ve varyasyonlar kontrol edilir.

Hoca, kuralları zamanında bu araçların nasıl adlandırıldığına bakarak çıkardığını, ayrıntılara fazla girmiş olabileceğini, ama asıl özümsenmesi gerekenin kalıp düzeyindeki işler olduğunu söyledi.

10.8 Çalıştırma sonucu

Program çalıştırıldığında konsol çıktısı (ekrandan, 01:48:30) şöyledir (ilk satırdaki araç adı kaydırıldığı için ekranda görünmüyor; hoca bunu "Honda Civic Sedan" diye okudu):

```
Engine: Petroleum 1600cc 125HP
Transmission: 6 gears automatic transmission
```

Yani benzinli, 1600 cc, 125 beygir gücünde motor ve altı ileri otomatik vites. "1.6L" token'ı 1600 cc'ye, "125PS" token'ı 125 beygir gücüne, "6AT" token'ı 6 vitesli otomatik şanzımana dönüşmüştür.

10.9 Esneklik ve tehdit

Hocaya göre bu kadar çıktı için bu kadar kod yazılmadı; önemli olan esnekliktir. Yeni bir marka satılacağında tek yapılması gereken, CarBuilder'ın yeni bir kardeş alt sınıfını (örneğin FordBuilder) eklemektir. CarDirector'da ve CarBuilder'da yapılması gereken ek bir şey yoktur.

> **Hocanın vurgusu:** Bir şey eklemek için yalnız bir sınıf eklemek ve başka hiçbir yerde değişiklik yapmamak, esnekliğin varabileceği en üst nokta ve en rahat bakım kolaylığıdır. Bu amaçla başka yerlere dokunmak gerekiyorsa, tasarım yeterince esnek yapılamamış demektir.

CarDirector'ın kodunda markayla ilgili hiçbir şey görünmez; yalnız aracın bileşenleri olan motor, vites ve kasa geçer. Tehdit, yeni bir alt parçanın çıkmasıdır: bu durumda farklı düzenlemeler gerekir ve o alt parça bazı marka-modellerde bulunmayabilir. Hocaya göre araçlarda ne olduğu artık bellidir; olsa olsa otonom sürüş özelliğine ilişkin bir bileşen gelebilir.

Örnek Sorular

Soru 1 (ekrandan, 00:20:00): Belge çerçevesi probleminde Factory Method kalıbı yerine başka hangi seçenekler kullanılabilir? Zayıf yönleri nelerdir?

> **Cevap (derste verildi):** Java'da generic sınıflar (`class Document<DocType>`), C++'taki template sınıflara benzer biçimde kullanılabilir. Zayıf yönü dile özel olmasıdır; uygulamanın taşınabilirliğini azaltır ve başka dile uyarlamak, dillerin farklı tasarım kararları nedeniyle zordur. Slayttaki "Başka?" sorusunu hoca açık bıraktı ve öğrencilerden geri bildirim istedi.

Soru 2 (derste sözlü, 00:32:14): Abstract Factory ve Factory Method kalıpları birbirine karıştırılabilir. Bir problemde hangisinin kullanılacağına karar verirken hatırlanması gereken en önemli etken nedir?

> **Cevap (derste verildi):** Abstract Factory'de ürün aileleri ve bu ailelerin üyeleri vardır (örneğin pencere yöneticisi aileleri ve onların radio button, check button, input box üyeleri). Factory Method'da aile hiyerarşisi yoktur; tek bir ürün hiyerarşisi için nesne oluşturma kararı alt sınıflara bırakılır.

Soru 3 (ekrandan, 01:00:00; derste sözlü, 01:02:20): RTF okuyucu örneğinde belgelere gömülü video desteği eklenmek istenirse Builder çözümünde ne olur?

> **Cevap (derste verildi):** Soyut TextConverter'a ConvertVideo metodu eklenmelidir. Bu metot eklenince, video bileşeninin anlamsız olduğu ASCIIConverter, TeXConverter ve TextWidgetConverter da metodu (boş gövdeyle) gerçeklemek zorunda kalır, yoksa derleme hatası olur. RTFReader.ParseRTF'teki switch'e de yeni bir case eklenir. Bu, kalıbın iki eleştirisini gösterir: switch-case tam ortadan kalkmamıştır ve alt sınıflarda anlamı olmayan metotlar boş gövdeli kalır.

Soru 4 (ekrandan, 01:10:00): Builder kalıbının yapısında switch-case görünmüyor. Gerçek bir gerçeklemede switch-case nerede ortaya çıkar ve nerede olması tercih edilir?

> **Cevap (derste verildi):** Director.construct'taki "for all objects in structure" döngüsündeki nesneler buildPart'a aktarılırken "parça şu ise şunu yap" denmek zorunda kalınacağı için switch-case ya ConcreteBuilder.buildPart içinde ya da Director.construct içinde ortaya çıkar. Somut builder sayısı fazla, Director tek olduğu için Director.construct içinde tek bir switch-case kullanmak, birden fazla somut builder içinde switch-case kullanmaktan daha iyidir.

Soru 5 (derste sözlü, 01:17:05): Builder kalıbını önceki factory kalıplarından ayıran nedir?

> **Cevap (derste verildi):** Builder'da ürünün alt parçalarının oluşturulup ürüne katılması üzerinde daha fazla denetim vardır. Director, Builder'a hangi işlemi hangi sırada yapacağını söyler; isterse bazı parçaları göndermeyip bir yarı mamul oluşturabilir. Factory kalıplarında ise istemci "bana şu üründen ver" der ve ürünü bütün olarak alır.

Soru 6 (derste sözlü, 01:40:02): Otomobil bayisi örneğinde CarBuilder.getResult() neden ürünün kendisini değil bir klonunu döndürüyor? Neden ayrıca protected bir getProduct() var?

> **Cevap (derste verildi):** Üye alan olan referans doğrudan döndürülürse, setter olmasa bile dışarıdan bu nesne üzerinde değişiklik yapılabilir (olmayan bir motor ya da vites atanabilir) ve builder'ın ürünü bozulur. Klon döndürülürse klondaki değişiklik asıl ürünü etkilemez. getProduct() ise alt sınıfların (HondaBuilder, RenaultBuilder)

ürüne erişip parçaları atayabilmesi için vardır ve dışarıdan erişilemesin diye protected'dır.

Soru 7 (ekrandan, 01:36:00, 01:45:40): `new HondaBuilder("Civic Sedan")` ile elemanlar `{"Engine\t125PS", "Engine\t1.6L", "Gear\t6AT"}` verilip construct ve report çağrılırsa motor ve vites bilgisi nasıl oluşur?

> **Cevap (derste verildi, ekrandan 01:48:30):** İlk Engine token'ında motor henüz yoksa varsayılan motor (benzinli, 1150 cc, 60 PS) oluşturulur; "125PS" içinde "PS" geçtiği için güç 125 olur. "1.6L" token'ı "L" içerdiği için motor hacmi olarak işlenir ve 1600 cc olur. "6AT" token'ı buildTransmission'a gider ve 6 ileri otomatik vites olarak yorumlanır. Çıktı: `Engine: Petroleum 1600cc 125HP` ve `Transmission: 6 gears automatic transmission`.

Dersin Toparlaması

Bu derste iki yaratımsal kalıp işlendi. Factory Method, nesne oluşturma arayüzünü soyut bir üst sınıfta (Creator) tutup hangi somut sınıftan nesne oluşturulacağı kararını alt sınıflara (ConcreteCreator) bırakır. Belge çerçevesi örneğinde Application.NewDocument() gibi somut metotlar ortak akışı yürütür, akışın farklılaşan noktasında soyut CreateDocument() çağrılır. Çizim uygulaması örneği, kalıbın oluşturmanın ötesinde, sorumluluğun hangi yardımcı alt sınıfa iletileceği bilgisini yerelleştirmek için de kullanılabileceğini gösterdi. E-ticaret örneğinde Store.makeOrder() sabit sipariş adımlarını yürütürken, createOrder() mağaza türüne göre Electronics ya da Books siparişi oluşturur. Abstract Factory ile farkı, Factory Method'da ürün ailelerinin olmamasıdır.

Builder ise karmaşık bir nesnenin oluşturulmasını gösteriminden ayırır: Director oluşturma adımlarını yönetir, soyut Builder parçaları oluşturma arayüzünü sunar, ConcreteBuilder'lar her gösterim için parçaları oluşturup birleştirir ve sonucu bir getter ile verir. GoF'un RTF okuyucu örneği kalıbın gücünü ama aynı zamanda iki zayıflığını (kalan switch-case ve boş gövdeli metotlar) gösterdi. Hocanın otomobil bayisi örneğinde yeni bir marka yalnız yeni bir CarBuilder alt sınıfıyla eklenebilir; getResult()'un klon döndürmesi ürünün dışarıdan bozulmasını engeller. Builder'ı factory kalıplarından ayıran şey, ürünün parçalarının oluşturulma süreci üzerindeki daha fazla denetimdir.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.